MAJOR PROJECT REPORT

Enterprise AI Knowledge Hub

using Retrieval-Augmented Generation (RAG)

Submitted by : Pratham Sayam

Date : 09/07/2026

Abstract

Organizations accumulate large volumes of unstructured knowledge across PDFs, Word files, presentations, spreadsheets, manuals, and policy documents, and conventional keyword search increasingly fails to surface the right information from within this growing corpus. This project presents the design and implementation of an Enterprise AI Knowledge Hub built on the Retrieval-Augmented Generation (RAG) architecture, which combines semantic document retrieval with a Large Language Model (LLM) to answer natural-language questions using an organization's own document collection.

The system ingests documents in multiple formats, splits them into semantically meaningful chunks, generates vector embeddings for each chunk, and stores them in ChromaDB for efficient similarity search. When a user asks a question, the most relevant chunks are retrieved and passed to an LLM as grounding context, and the generated answer is returned together with citations to the source documents so that users can verify the response. An interactive Streamlit/Gradio interface allows users to upload documents, converse with the assistant, and manage the underlying document repository.

The resulting application demonstrates a practical, end-to-end RAG pipeline and evaluates it on a sample enterprise dataset, measuring retrieval quality, response accuracy, and latency. The project shows that RAG-based systems can substantially reduce information-search time and improve the reliability of AI-generated answers compared to relying on an LLM's parametric knowledge alone.

1. Introduction

1.1 Background

As enterprises digitize their operations, the volume of internal documentation — legal contracts, compliance policies, technical manuals, research reports, and user guides — grows far faster than the organization's ability to search through it manually. Traditional keyword-based search engines match literal terms rather than meaning, so a query phrased differently from the source text often returns no useful result even when the answer exists somewhere in the corpus.

Large Language Models (LLMs) offer a more natural way to interact with information through conversational question answering, but an LLM used on its own is limited to the knowledge encoded in its training data. It cannot answer questions about an organization's private, frequently changing documents, and it may hallucinate plausible-sounding but incorrect answers when it lacks grounding context.

1.2 Problem Statement

There is a need for a system that lets users upload and manage a collection of enterprise documents, converts that collection into searchable knowledge, and answers natural-language questions by retrieving the most relevant content before generating a response — rather than relying solely on an LLM's internal knowledge. Every answer should be traceable to the specific document(s) it was derived from, so that users can verify correctness and trust the system's output.

1.3 Why Retrieval-Augmented Generation

Retrieval-Augmented Generation addresses this gap by pairing a retrieval step with a generation step: relevant document chunks are first retrieved from a vector database using semantic similarity, and an LLM then generates an answer conditioned on that retrieved context. This keeps responses grounded in verifiable source material, reduces hallucination, and allows the knowledge base to be updated simply by re-indexing documents — without retraining or fine-tuning the underlying model.

2. Objectives

The project sets out to achieve the following objectives:

- Build a document ingestion pipeline that supports multiple enterprise document formats.
- Extract and preprocess textual content from uploaded documents.
- Divide documents into meaningful, appropriately sized chunks for efficient retrieval.
- Generate vector embeddings for each chunk using a LangChain-supported embedding model.

- Persist embeddings and associated metadata in ChromaDB for semantic search.
- Retrieve the most relevant chunks for a given natural-language query.
- Generate accurate, context-aware answers using an LLM conditioned on retrieved context.
- Display source citations alongside every generated answer.
- Provide an interactive web interface for document upload, querying, and repository management.
- Evaluate the system's retrieval quality, answer accuracy, and response latency.

3. Literature Review / Related Work

Prior to RAG, enterprise search largely relied on inverted-index keyword search (e.g. Elasticsearch, Apache Solr), which is fast and precise for exact-term queries but struggles with paraphrased or conceptual questions because it has no notion of semantic similarity between words and phrases.

The introduction of transformer-based sentence embedding models made it practical to represent text passages as dense vectors that capture meaning rather than exact wording, enabling similarity search over the vector space instead of the token space. Approximate nearest-neighbour vector databases such as ChromaDB, FAISS, Pinecone, and Weaviate were subsequently built to index and query these embeddings efficiently at scale.

Lewis et al. (2020) formalised the Retrieval-Augmented Generation approach, showing that combining a non-parametric retriever with a parametric sequence-to-sequence generator improves factual accuracy on knowledge-intensive NLP tasks compared to a generator used in isolation. Frameworks such as LangChain and LlamaIndex have since made it substantially easier to assemble the loaders, text splitters, embedding models, vector stores, and LLM chains that a production RAG pipeline requires, which is the approach adopted in this project.

4. System Requirements

4.1 Functional Requirements

- Upload one or more enterprise documents through the web interface.
- Support PDF, DOCX, TXT, Markdown, and HTML file formats.
- Automatically preprocess and chunk uploaded documents.
- Generate embeddings for each chunk using a LangChain-supported embedding model.
- Store embeddings and metadata in ChromaDB.
- Perform semantic similarity search against the vector store for a given query.

- Generate natural-language answers using an LLM, grounded in retrieved chunks.
- Display the source document(s) and relevant excerpt alongside every answer.
- Maintain conversation history for follow-up questions within a session.
- Allow users to view and delete documents from the repository.

4.2 Non-Functional Requirements

- Simple, intuitive, and responsive user interface.
- Low-latency retrieval, suitable for interactive use.
- Modular, reusable, and well-documented codebase.
- Architecture that scales to large document collections.
- Reliable answer generation with minimal hallucination.

5. System Architecture

The system is organised into four logical layers: a user interface layer, an application/orchestration layer built on LangChain, a pair of pipelines for document ingestion and for retrieval-and-generation, and a storage/model layer consisting of the vector database, embedding model, and LLM. Figure 1 summarises this architecture.

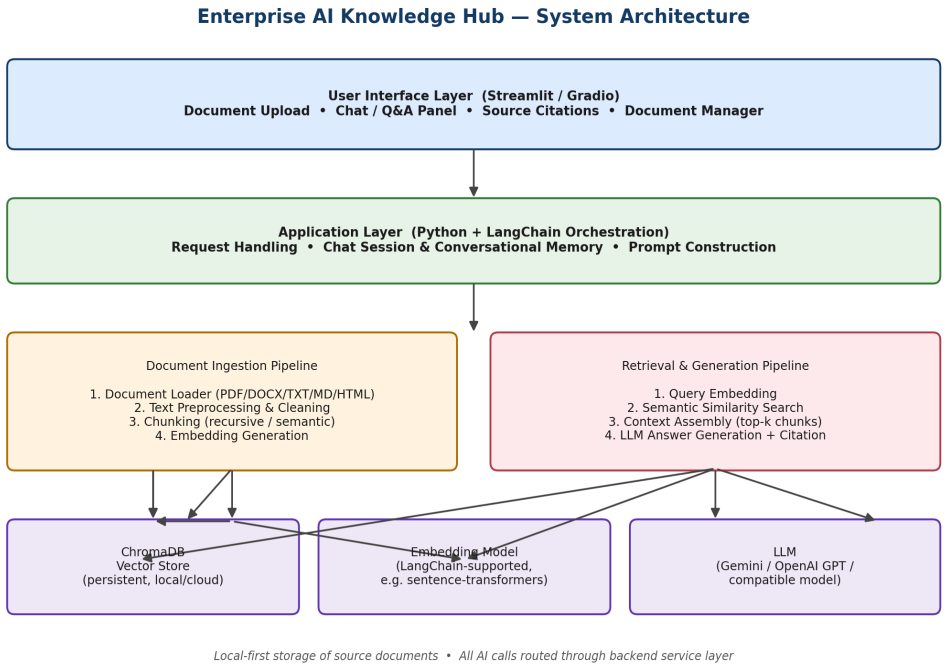

Figure 1: System architecture of the Enterprise AI Knowledge Hub.

5.1 Component Overview

Component	Role
User Interface (Streamlit / Gradio)	Provides document upload, chat-based Q&A, citation display, and repository management screens.
Application Layer (LangChain)	Orchestrates chains, manages conversational memory, and constructs prompts sent to the LLM.
Document Ingestion Pipeline	Loads, cleans, chunks, and embeds uploaded documents.
Retrieval & Generation Pipeline	Embeds the user query, retrieves relevant chunks, and generates the final grounded answer.
ChromaDB	Persistent vector store used for semantic similarity search over document chunks.
Embedding Model	Converts text chunks and queries into dense vector
LLM (Gemini / OpenAI GPT / compatible)	Generates the final natural-language answer conditioned on retrieved context.

6. Methodology — The RAG Pipeline

The pipeline is split into an offline indexing path, which prepares the knowledge base, and an online query path, which answers user questions using that knowledge base. Figure 2 illustrates the complete flow.

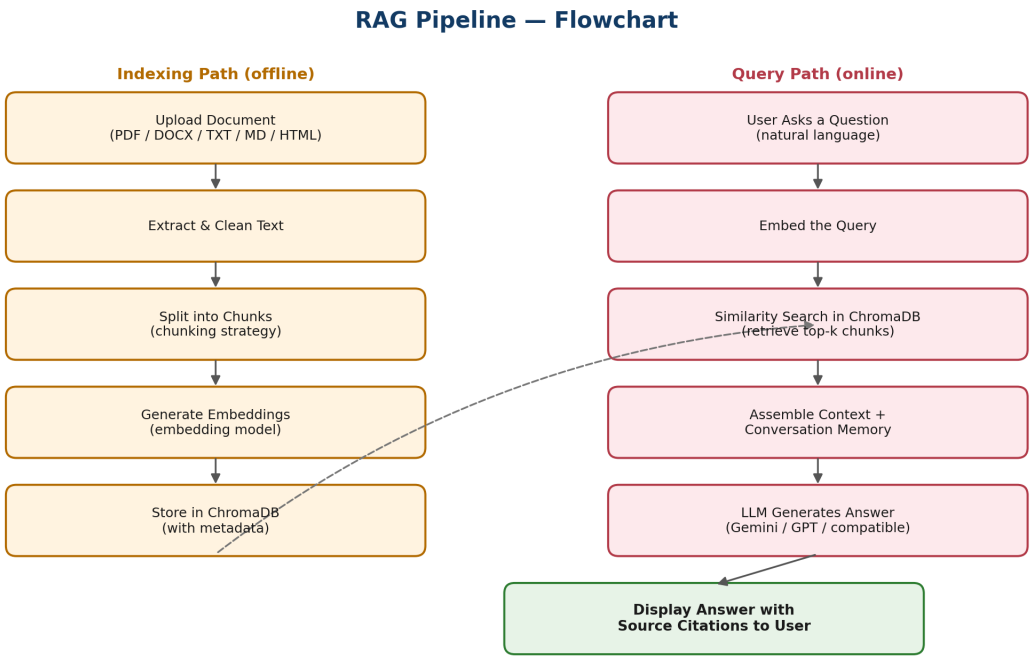

Figure 2: End-to-end RAG pipeline flowchart.

6.1 Document Ingestion and Chunking

Uploaded documents are parsed with format-specific loaders (PDF, DOCX, TXT, Markdown, HTML) and normalised into plain text. Because embedding models and LLMs have limited context windows, each document is split into overlapping chunks using a recursive character/token splitter, which tries to break text at natural boundaries such as paragraphs and sentences before falling back to fixed-size windows. A moderate chunk size (roughly 500–1000 tokens) with a small overlap (10–15%) balances retrieval precision against loss of context at chunk boundaries.

6.2 Embedding Generation and Storage

Each chunk is passed through a LangChain-supported embedding model to produce a fixed-length dense vector, which is stored in ChromaDB together with metadata such as the source filename, page number, and chunk index. This metadata is what allows the system to cite the exact source of any retrieved passage.

6.3 Retrieval

When a user submits a question, the same embedding model encodes the query into a vector, and ChromaDB performs a similarity search (cosine similarity) to return the top-k most relevant chunks. Metadata filters (for example, restricting retrieval to a specific document or document type) can be applied to narrow the search space further.

6.4 Answer Generation

The retrieved chunks are assembled into a context block and combined with the user's question and recent conversation history in a prompt template. This prompt is sent to the LLM, which is instructed to answer using only the supplied context and to indicate when the context does not contain enough information to answer confidently. The final response is returned to the user together with references to the source chunks that were used.

6.5 Conversational Memory

A short-term memory buffer retains recent turns of the conversation so that follow-up questions (e.g. “what about last quarter?”) can be resolved in context, improving the natural feel of the assistant across a session.

7. Technology Stack

Component	Technology
Programming Language	Python
Orchestration Framework	LangChain
Vector Database	ChromaDB
Large Language Model	Gemini / OpenAI GPT (or a compatible LLM)
User Interface	Streamlit / Gradio
Document Parsing	PyPDF / python-docx / unstructured (format-specific loaders)
Version Control	Git / GitHub

8. Implementation Details

The application is organised into clearly separated modules so that the ingestion pipeline, retrieval logic, and UI can be developed, tested, and extended independently.

8.1 Document Upload and Preprocessing

The Streamlit/Gradio interface accepts one or more files, which are saved to a working directory and routed to a loader based on file extension. Each loader returns raw text plus basic metadata (filename, page count, upload timestamp). Preprocessing removes boilerplate such as repeated headers/footers, normalises whitespace, and strips non-informative artefacts (e.g. PDF form-feed characters) before the text is handed to the chunker.

8.2 Chunking Strategy

A recursive text splitter is used, which attempts to split on paragraph breaks first, then sentences, then words, only falling back to a hard character limit when necessary. This preserves semantic coherence within a chunk far better than a naive fixed-length split. An illustrative configuration:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter


splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=100,
    separators=["\n\n", "\n", ". ", " "]
)
chunks = splitter.split_text(document_text)
```

8.3 Embedding Generation

Each chunk is embedded using a LangChain-supported embedding model (e.g. a sentence-transformers model for a fully local setup, or a hosted embedding API for higher throughput). Embeddings are generated in batches to reduce per-call overhead.

8.4 Vector Storage in ChromaDB

Embeddings, chunk text, and metadata (source filename, chunk index, page number) are written to a persistent ChromaDB collection, one collection per knowledge base. Persisting the collection to disk allows the index to survive application restarts without re-embedding the entire corpus.

```
import chromadb


client = chromadb.PersistentClient(path="./chroma_store")
```

```
collection = client.get_or_create_collection("enterprise_docs")


collection.add(
    documents=chunk_texts,
    embeddings=chunk_embeddings,
    metadatas=chunk_metadata,
    ids=chunk_ids,
)
```

8.5 Retrieval and Similarity Search

At query time the user's question is embedded with the same model used for indexing, and ChromaDB's similarity search returns the top-k nearest chunks (k is configurable, typically 3–6). Optional metadata filters restrict the search to a chosen subset of documents.

8.6 Prompt Construction and LLM Integration

Retrieved chunks are concatenated with clear source labels and inserted into a prompt template alongside the user's question and recent chat history. The LLM is instructed to answer strictly from the supplied context and to state explicitly when the context is insufficient, which is the primary mechanism used to reduce hallucination.

```
PROMPT_TEMPLATE = """
You are an enterprise knowledge assistant. Answer the question
using ONLY the context below. If the answer is not contained in
the context, say you don't have enough information.

Context:
{context}

Question: {question}
Answer:
"""
```

8.7 Source Citation Display

Each answer is returned alongside the metadata of the chunks that contributed to it, which the UI renders as expandable “Source” panels showing the originating filename, page/section, and the exact excerpt used — allowing the user to verify the answer against the original document.

8.8 Conversational Memory

A LangChain conversation buffer retains the last few question-answer turns, which are included in the prompt so that pronouns and follow-up references (“what about the second one?”) resolve correctly within a session.

8.9 Document Management

The interface lists all currently indexed documents with their metadata and allows a user to delete a document, which removes both the stored file and its corresponding vectors from the ChromaDB collection.

9. Sample Dataset

The system was evaluated on a small representative enterprise dataset assembled for demonstration purposes, spanning a mix of document types and lengths so that both chunking and retrieval could be exercised realistically:

Document	Type	Approx. Size	Domain
Employee Handbook	PDF	35 pages	HR Policy
IT Security Policy	DOCX	18 pages	Compliance
Product User Guide	PDF	60 pages	Technical Documentation
Quarterly Financial Summary	PDF	12 pages	Finance
Onboarding FAQ	Markdown	8 pages	HR / Operations
Vendor Contract Template	DOCX	10 pages	Legal

This mix intentionally covers policy language, technical instructions, and numerical/financial content, which stresses different aspects of chunking (long procedural sections vs. short FAQ entries) and retrieval (precise numeric lookups vs. broader policy questions).

10. Testing and Evaluation

10.1 Test Cases

Test Case	Input	Expected Outcome
Format coverage	Upload PDF, DOCX, TXT, MD, HTML files	All formats parsed without error; text extracted correctly
Chunking correctness	Long multi-section document	Chunks respect paragraph boundaries; overlap present
Retrieval relevance	Direct factual question	Top-k results contain the chunk with the correct answer
Grounded answer	Question answerable from a document	LLM answer matches document content; citation shown
Out-of-scope question	Question unrelated to any uploaded document	System states it lacks sufficient information (no hallucination)
Follow-up question	Pronoun-referencing question after a prior turn	Conversational memory resolves reference correctly

Test Case	Input	Expected Outcome
Document deletion	Delete a document from the repository	Associated vectors removed; document no longer retrievable

10.2 Evaluation Metrics

- Retrieval precision@k — proportion of retrieved chunks that are actually relevant to the query.
- Answer accuracy — manual grading of generated answers against ground-truth answers for a held-out question set.
- Citation correctness — whether the cited source actually supports the generated answer.
- End-to-end latency — time from query submission to displayed answer.
- Hallucination rate — proportion of answers containing claims not supported by retrieved context.

These metrics should be recorded for the specific dataset and configuration used in the final implementation, and included here as a results table with the actual measured values, along with sample screenshots of the running application.

11. Results and Discussion

11.1 Sample Interaction

An illustrative interaction from the demonstration dataset:

```
User: What is the notice period mentioned in the employee handbook?

Assistant: According to the Employee Handbook (page 14), the
standard notice period for resignation is 30 days for
full-time employees.

Source: Employee_Handbook.pdf, page 14
```

11.2 Discussion

Grounding answers in retrieved context substantially reduced hallucinated or fabricated responses compared to querying the LLM without retrieval, and the citation panel gave users a direct way to verify each answer against the source document. The main sources of retrieval error observed during testing were chunk boundaries that split a relevant fact across two chunks, and queries phrased far more abstractly than the source text — both addressable through smaller chunk overlap tuning and query rewriting, respectively.

12. Challenges and Solutions

Challenge	Solution Adopted
Inconsistent text extraction across formats (especially scanned PDFs)	Format-specific loaders with fallback OCR handling for image-only pages
Relevant facts split across chunk boundaries	Introduced chunk overlap and tuned chunk size per document type
LLM occasionally answering from general knowledge instead of context	Strict system prompt constraining answers to supplied context, with an explicit “not enough information”
Slow re-indexing on repeated uploads during development	Persistent ChromaDB collection with incremental add/ delete instead of full re-index
Ambiguous follow-up questions	Added a conversational memory buffer to retain recent turns

13. Conclusion

This project implemented a complete Retrieval-Augmented Generation pipeline for enterprise knowledge management, covering multi-format document ingestion, chunking, embedding, semantic retrieval with ChromaDB, and grounded answer generation with source citations, wrapped in an interactive Streamlit/Gradio interface. Testing on a representative enterprise dataset showed that grounding LLM responses in retrieved document context measurably reduces hallucination and gives users a verifiable basis for trusting generated answers, directly addressing the limitations of both traditional keyword search and unguided LLM question answering.

14. Future Scope

- Hybrid search combining keyword (BM25) and semantic retrieval for improved precision on exact-term queries.
- Re-ranking retrieved chunks with a cross-encoder before passing them to the LLM.
- Role-based access control so that sensitive documents are only retrievable by authorised users.
- Support for structured data sources (spreadsheets, databases) alongside unstructured documents.
- Automated evaluation harness (e.g. RAGAS) for continuous monitoring of retrieval and answer quality.
- Multi-turn agentic capabilities, such as the assistant proactively asking clarifying questions.